

# HIGH-LEVEL DESIGN

## User Posts & Albums Viewer

Client-side web application using HTML, CSS, JavaScript and jQuery

|                  |                                     |
|------------------|-------------------------------------|
| Document Version | 0.1                                 |
| Status           | Draft - based on confirmed RUD v1.1 |
| Prepared By      | Atul Sharma                         |
| Data Source      | JSONPlaceholder REST API            |

### 1. Purpose

This High-Level Design defines the major components, responsibilities, external interface, and high-level request/data flows for the User Posts & Albums Viewer. It is derived from the confirmed requirements recorded in RUD v1.1 and is intentionally kept above implementation-level detail.

### 2. Scope and Design Boundaries

- The solution is a client-side / presentation-layer web application.
- HTML and CSS provide the page structure and visual presentation.
- JavaScript with jQuery provides client-side interaction, API communication, response processing, and DOM updates.
- JSONPlaceholder is the only external data source currently in scope.
- No custom backend, database, authentication, or persistent storage is part of this design.

### 3. Architecture Overview

The system follows a simple client-to-external-API architecture. The browser hosts the complete application logic. The external REST API supplies user, post, and album data.



Figure 1. High-level component and interaction view

## 4. Major Components

| Component                     | Responsibility                                                                                               | Key technology                   |
|-------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------|
| Presentation Layer            | Renders users, selected-user area, Posts/Albums controls, content, loading and error states.                 | HTML5 + CSS3                     |
| Client-side Application Logic | Handles user interaction, API requests, HTTP success checks, JSON processing and coordination of UI updates. | JavaScript (ES6+) + jQuery 4.0.0 |
| External API                  | Provides user, post and album records through REST-style GET endpoints.                                      | JSONPlaceholder                  |

## 5. High-Level Data / Request Flow

- 1. Initial load:** The application requests the user collection from GET /users.
- 2. User rendering:** Returned users are rendered using first name + last name.
- 3. Selection:** The user selects one displayed user.
- 4. User-specific retrieval:** The application retrieves that user's posts and albums using the selected user ID.
- 5. Separate presentation:** The UI exposes separate Posts and Albums options; the selected option controls the displayed content area.
- 6. Feedback states:** Loading is shown during retrieval; an error state is shown when an API request fails.

## 6. External API Interface at HLD Level

| Purpose         | HTTP Method | Endpoint               | Used when                  |
|-----------------|-------------|------------------------|----------------------------|
| Retrieve users  | GET         | /users                 | Application initialization |
| Retrieve posts  | GET         | /users/{userId}/posts  | After a user is selected   |
| Retrieve albums | GET         | /users/{userId}/albums | After a user is selected   |

The client application treats these endpoints as read-only data sources for this assignment. Detailed response schemas and endpoint-specific behavior are maintained in the API Specification.

## 7. Component Responsibilities and Interactions

### 7.1 Presentation Layer

- Displays the user list with first name and last name.
- Provides the user selection control.
- Provides separate Posts and Albums controls/views in the selected-user area.
- Displays loading and error feedback to the user.

### 7.2 Client-side Application Logic

- Initiates API requests using jQuery-based HTTP/AJAX operations.
- Checks whether API responses indicate success before processing the returned data.
- Processes JSON responses and passes the resulting data to the rendering layer.
- Tracks the selected user and coordinates the displayed Posts or Albums view.

### 7.3 External API Boundary

- Provides the user, post, and album records needed by the UI.
- No application-owned business database or service layer is introduced in this assignment.

## 8. Interaction Scenarios

| Scenario                 | Client-side flow                                              | Result                       |
|--------------------------|---------------------------------------------------------------|------------------------------|
| Initial application load | UI initialized → GET /users → parse JSON → render user list   | Users visible                |
| User selection           | Selection event → capture user ID → request posts + albums    | Selected-user data available |
| Posts view               | Posts option selected → render posts in shared content area   | Only posts are shown         |
| Albums view              | Albums option selected → render albums in shared content area | Only albums are shown        |
| API failure              | Request fails or returns non-success → error handling         | User-facing error message    |

## 9. UI State Model

| State                   | When it applies                                  | Expected UI behavior                          |
|-------------------------|--------------------------------------------------|-----------------------------------------------|
| Initial / Users Loading | During initial GET /users                        | Show loading indication in the users area     |
| Users Loaded            | Users successfully retrieved                     | Render first name + last name list            |
| User Detail Loading     | Posts/albums are being retrieved after selection | Show loading indication in selected-user area |
| Posts View              | Posts option selected                            | Render posts only                             |
| Albums View             | Albums option selected                           | Render albums only                            |
| Error                   | Relevant API request fails                       | Show an appropriate user-facing error state   |

## 10. Cross-Cutting Design Considerations

### 10.1 Error Handling

- Non-success HTTP responses are treated as request failures at the client-logic layer.
- Network/request failures are surfaced to the UI through an error state.
- The user should not be left with an unexplained loading state when a request fails.

### 10.2 Loading and Feedback

- Loading feedback is associated with the request currently in progress.
- The selected-user content area should communicate that user-specific data is being fetched.

### 10.3 Maintainability

- API communication, interaction handling, and DOM rendering should remain logically separated where practical.
- Endpoint construction should use the selected user ID rather than hard-coded user-specific URLs.
- The design avoids introducing unnecessary abstraction because the assignment has a small scope.

### 10.4 Security / Trust Boundary

- JSONPlaceholder is an external data source and should be treated as untrusted input for rendering purposes.
- User-provided or externally returned text should be inserted using safe text-oriented DOM operations where practical, rather than unnecessary raw HTML injection.
- No application authentication or authorization is included because it is outside the confirmed assignment scope.

## 11. Performance Considerations

- The initial request retrieves only the user collection required to populate the directory.
- Posts and albums are retrieved for the selected user rather than loading unrelated users' data.

- Independent user-detail requests may be coordinated concurrently during implementation where appropriate; this is an implementation choice rather than an architectural requirement.

## 12. Deployment Assumption

The design assumes a static/client-side deployment capable of serving the HTML, CSS and JavaScript assets to a modern browser. The application communicates directly with JSONPlaceholder over HTTPS. No server-side application component is defined in this HLD.

## 13. HLD-to-LLD Boundary

The following items are intentionally deferred to Low-Level Design and implementation documentation:

- Exact JavaScript/jQuery function names and module/file structure
- Detailed event-handler implementation
- Exact DOM element hierarchy and selectors
- Detailed AJAX option objects and Promise/callback handling
- Exact loading/error component markup
- Detailed validation and rendering procedures

## 14. Traceability to Confirmed Requirements

| Requirement               | HLD coverage                                                          |
|---------------------------|-----------------------------------------------------------------------|
| Retrieve users            | Presentation and client-logic components communicate with GET /users. |
| Display first + last name | Presentation layer owns user-list rendering.                          |
| Select user               | Presentation event flows into client-side application logic.          |
| Retrieve posts            | Client logic calls GET /users/{userId}/posts.                         |
| Retrieve albums           | Client logic calls GET /users/{userId}/albums.                        |
| Separate Posts and Albums | UI state model defines distinct Posts and Albums views.               |
| Loading state             | Cross-cutting feedback/state model includes loading states.           |
| Error handling            | Client logic detects failures and presentation layer displays errors. |

## 15. Assumptions / Design Decisions

- The application is intentionally modeled as client-side only, consistent with the confirmed assignment scope.
- JSONPlaceholder is the only data source; no persistence layer is introduced.
- The UI remains simple and feature-focused because no strict visual design requirement was specified; Posts and Albums are separate user-selectable views in the same content area.